Sistema Caché con Geolocalización

Este proyecto implementa un ecosistema completo compuesto por múltiples servicios distribuidos, cuyo objetivo es interceptar tráfico DNS, redirigirlo hacia caches zonales optimizadas por geolocalización, exponer APIs auxiliares y proporcionar una interfaz gráfica para la gestión de dominios, caché y autenticación.

Componentes del Proyecto

La solución se organiza en los siguientes componentes principales:

DNS Interceptor

El DNS Interceptor es una aplicación escrita en C que actúa como intermediario entre los clientes que realizan consultas DNS y las caches zonales distribuidas.
Su función es interceptar solicitudes DNS hacia dominios específicos y decidir cuál cache zonal debe atender la petición.

Funciones principales

- Interceptación de paquetes DNS.
- Se consulta a la API la IP de la caché más cercana, enviándole la IP del usuario.
- La API verifica en Firebase la ubicación del usuario y compara con la ubicación de las cachés.
- La API retorna la IP de la caché correspondiente.
- Se crea el paquete DNS con la respuesta obtenida desde la API.
- Se envía la respuesta al cliente.

Zonal Cache

La Zonal Cache es un servicio implementado en C, encargado de almacenar en disco los recursos cacheados para cada dominio, según las configuraciones definidas en Firebase.
Este componente es fundamental para mejorar la latencia, disponibilidad y eficiencia mediante políticas avanzadas de reemplazo de contenido.

Funcionalidades principales

1. Autenticación con la API

La caché debe enviar una API Key al comunicarse con la API, para asegurar la seguridad en la comunicación.

2. Recepción y análisis de la solicitud

- Recibe la petición del cliente.
- Identifica el dominio consultado.
- Solicita a la API el tipo de autenticación requerido para ese dominio o URL.

3. Validación de autenticación del usuario

Autenticación mediante API Key

- Las peticiones pueden estar protegidas mediante el header:
  x-api-key:
- Si la API Key es inválida o ausente, el acceso se deniega.

Autenticación mediante usuario / password

Si el dominio requiere autenticación por credenciales:

- La caché verifica si la petición incluye un session_id.
- Si el session_id es inválido o no existe, el usuario es redirigido al formulario de login en Vercel.
- La caché valida credenciales y sesiones comunicándose directamente con la UI.
- Toda petición subsecuente debe incluir un session_id válido.

4. Consulta del archivo en cache

Una vez autenticado el usuario, la Zonal Cache verifica si el archivo solicitado está en el disco.

Cada archivo respeta:

- Su TTL correspondiente.
- La política de reemplazo definida para el dominio.
- Si el archivo existe y no ha expirado, se retorna al cliente.

5. Consulta al servicio de origen

Si el recurso no está en cache o ha expirado:

- La caché solicita a la API la IP del dominio de origen.
- Realiza la solicitud al servicio correspondiente.

La respuesta:

- Se envía al usuario.
- Se almacena en disco según las reglas configuradas (TTL y política de reemplazo).

Algoritmos de reemplazo soportados

La Zonal Cache soporta los siguientes algoritmos, configurables desde Firebase por dominio:

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- FIFO (First-In First-Out)
- MRU (Most Recently Used)
- Random

REST API Simple

API para gestionar una lista de nombres en memoria, utilizando los métodos HTTP principales.

Endpoints

- GET /get_all
  Retorna todos los nombres.

- POST /add_element
  Agrega un nombre nuevo.
  Cuerpo: { "nombre": "" }

- PUT /update_element/:id Actualiza el nombre con el id especificado.
  Cuerpo: { "nombre": "<nuevo_nombre>" }

- DELETE /delete_element/:id Elimina el nombre con el id dado.

La API usa Express y mantiene los datos en una lista en memoria.

Apache Server

Componente servidor Apache que sirve una página HTML básica.
Esta página funciona como prueba del proyecto, mostrando "Hello World from Apache".

REST API Node.js

API en Node.js encargada de la comunicación con Firebase / Firestore para la UI y las caches zonales.

Funciones principales

Proveer endpoints para crear, modificar, eliminar o consultar configuraciones relacionadas con:

- Dominios
- URLs
- Políticas de cache
- Tamaños máximos de cache
- TTL por tipo de archivo
- Autenticación

Seguridad

Autenticación mediante API Keys. Cada cliente que realice llamadas a la API debe proporcionar su API Key válida para acceder a los endpoints.

UI (Frontend)

La UI es una aplicación web desarrollada con:

- React
- Vite
- TailwindCSS
- Asistencia de IA generativa

El acceso a las funcionalidades de la interfaz está asegurado mediante JSON Web Tokens (JWT) que se asignan cuando el usuario ingresa a la página.

Funciones principales

1. Registro de usuario

- Crear cuentas nuevas.
- Validación de dominio mediante registro TXT.

2. Administración de dominios

- Crear y eliminar dominios.
- Verificación mediante registro TXT.
    - Para esto se eligió usar el servicio Duck DNS. Esta herramienta fue seleccionada luego de una investigación por su facilidad de uso e implementación.
    - Para utilizar esta herramienta el usuario debe tener una cuenta creada en la página de Duck DNS, crear el dominio que quiere para verificar que está disponible, luego de verificar que está disponible crearlo tanto en la página de duck como en la interfaz, y para realizar la verificación mediante registro TXT debe mandar el registro al API que ofrece duck con los datos que solicita
- Gestión de metadatos asociados a cada dominio.

3. Administración de URLs

Para cada dominio registrado, permite:

- Crear, borrar y modificar URLs.
- Soporte para wildcards.
- Configuración por URL:
    - Tamaño máximo de cache.
    - Tipos de archivos permitidos (png, html, css, js, etc.).
    - TTL por tipo de archivo.
    - Política de reemplazo (LRU, LFU, FIFO, MRU, Random).

4. Mecanismos de autenticación

- API Key:
    - Crear y eliminar API Keys.
- Usuario / Password:
    - Crear, modificar y eliminar usuarios.
    - Asignar permisos a recursos cache.

5. Formulario de autenticación

- Utilizado por las caches zonales para validar usuarios y contraseñas.

6. Cerrar sesión

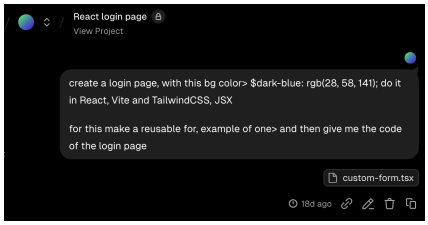
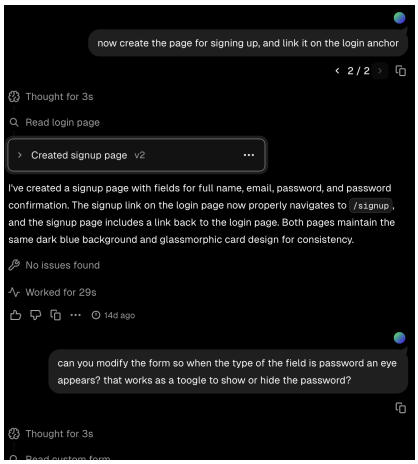
- Manejo de sesiones persistentes.

- Destrucción del token o sesión activa.

IA generativa utilizada

Para el desarrollo de la UI el equipo se apoyó en v0 de Vercel. Esta es una herramienta de IA generativa bastante reconocida que facilita la creación de código frontend (principalmente React y Tailwind CSS) a partir de prompts descriptivos, permitiendo previsualizar el resultado de forma inmediata. La selección de v0 se basó en su popularidad e integración con el stack del proyecto.

Parte de los props que se le enviaron fueron:

Prop	Props	Prop
		

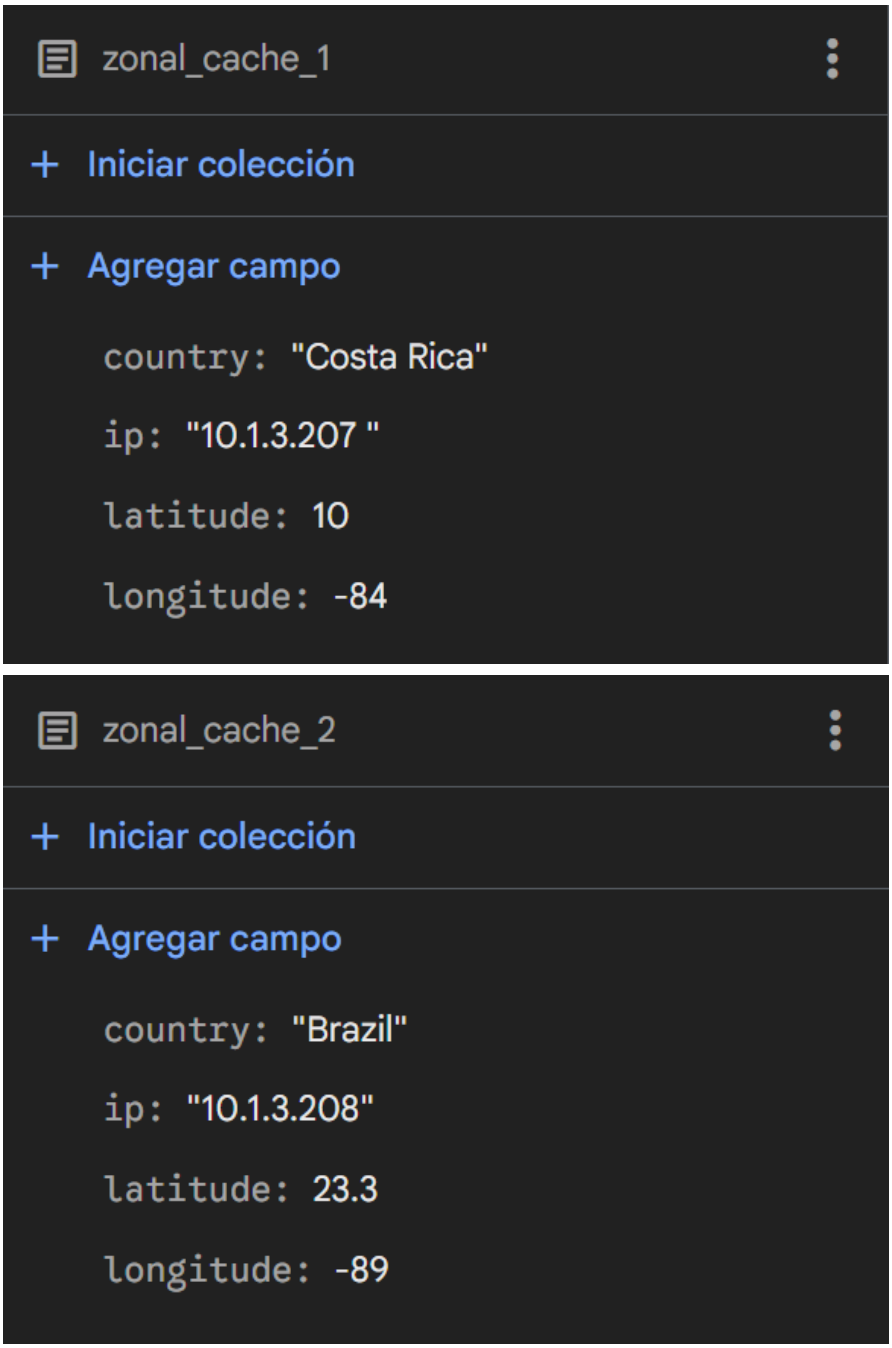
Opinión: Notamos que trabaja mejor con props en inglés y props que incluyan un ejemplo de código, así el código que genera sigue la forma de programación que uno le da y no es tan ineficiente. En este caso se reusaron varios componentes que ya se tenían por lo que facilitó mucho el trabajo dándole archivos en los que se pudiera basar. Si es importante antes de pedirle algo ya tener una idea de lo que uno necesita exactamente y luego de eso uno analizar qué partes de códigos o archivos se pueden simplificar, porque la herramienta si tiene mucho esa falla de no reutilizar tanto, pero además de eso con lo que fueron colores y el uso de Tailwind lo entendió bastante bien y facilitó mucho el trabajo porque hubieron muchos componentes y páginas que se parecían por lo que se podían reutilizar.

Pruebas Funcionales

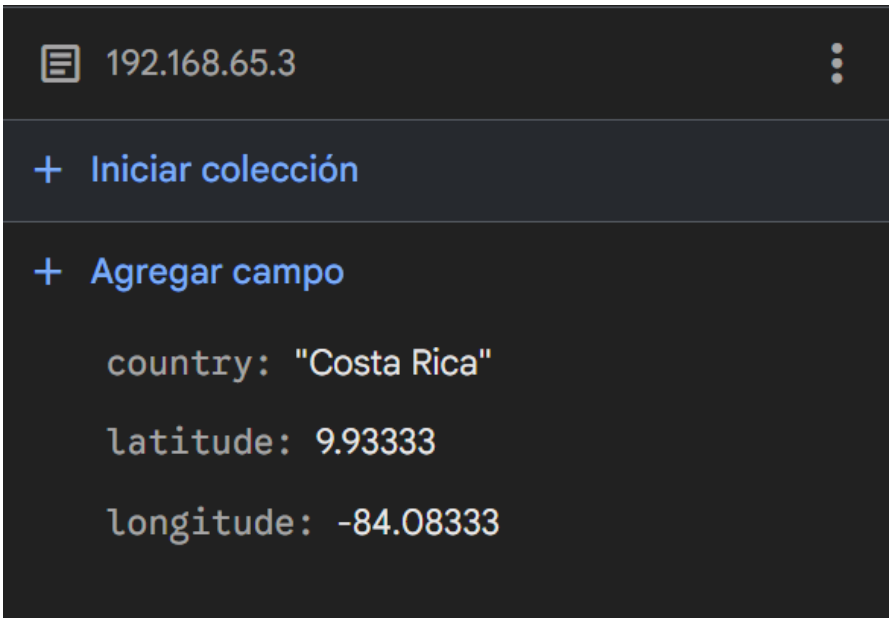
DNS Interceptor

Para verificar el correcto funcionamiento del DNS Interceptor, se utilizó `nslookup` como cliente de prueba.

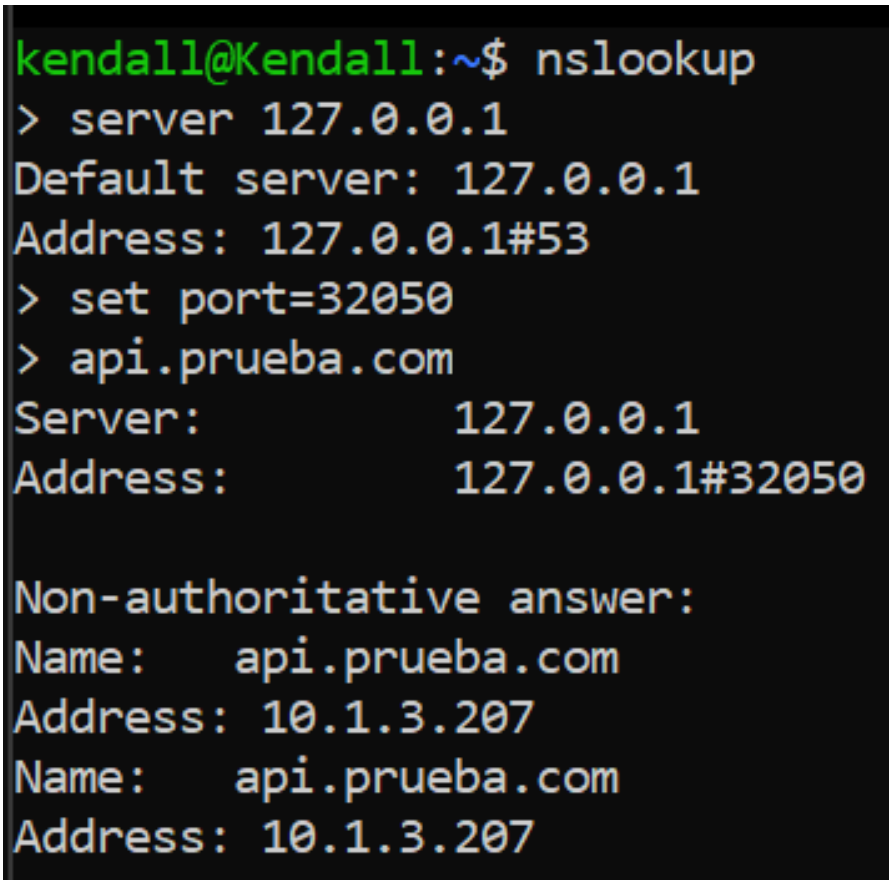
En Firebase se encuentran registradas dos cachés zonales, ubicadas en Costa Rica y Brasil, como se muestra en las siguientes imágenes:



A continuación, se muestra la ubicación de la IP del usuario, que corresponde a Costa Rica:



En la siguiente imagen se observa cómo se realiza la consulta desde el usuario y el DNS Interceptor retorna la IP `10.1.3.207`, correspondiente a la caché zonal más cercana en Costa Rica, funcionando correctamente:



Zonal Cache

Para comprobar el funcionamiento de la caché, se registró manualmente en el archivo hosts de la PC el dominio obtenido en Firebase, de forma que apuntara a 127.0.0.1:80 (localhost), donde se encuentra ejecutándose la caché.

Para validar el proceso de autenticación del usuario, se realizaron pruebas utilizando dos métodos: API-Key y User/Pass.

Para este proceso se utilizó el dominio p02redes.duckdns.org junto con los subdominios www y test.

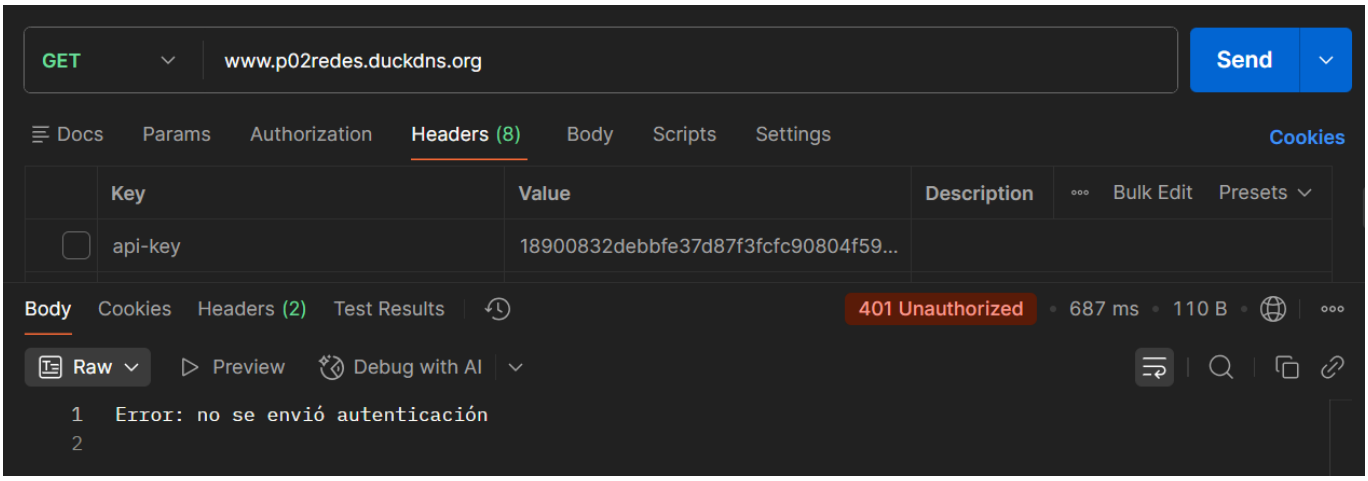
- El subdominio www utiliza autenticación mediante api key.
- El subdominio test utiliza autenticación por usuario y contraseña (userpass).

API-KEY

Para validar el flujo de autenticación mediante api key, se realizó la búsqueda del dominio correspondiente desde el navegador, utilizando el subdominio de pruebas (www), el cual, como se mencionó anteriormente, está configurado con autenticación tipo api key.

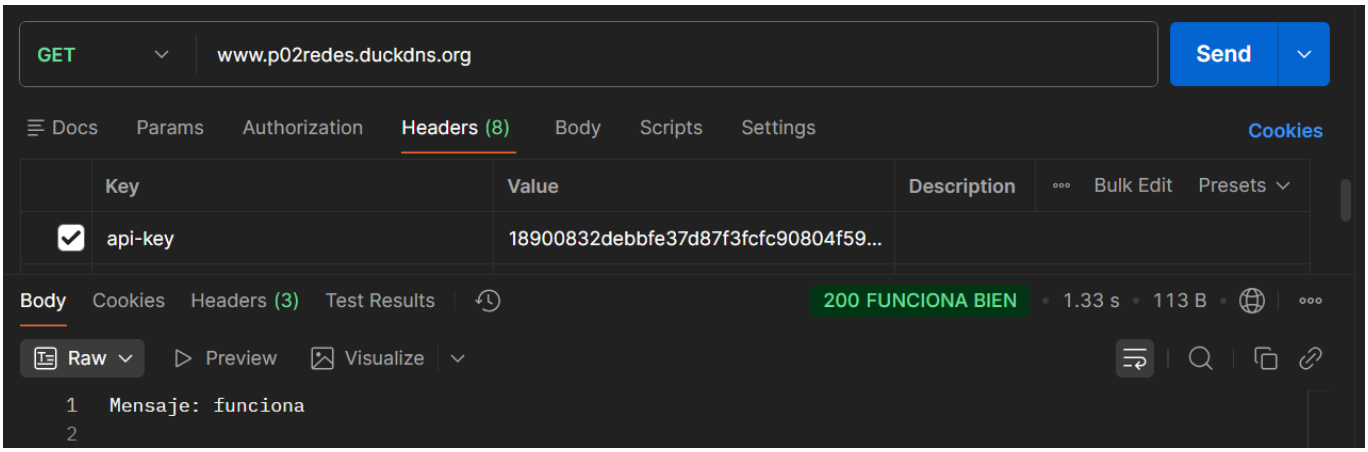
1. Solicitud sin API-Key

Se utilizó Postman como cliente para realizar la búsqueda del dominio en la caché sin enviar ningún API-Key. La caché respondió correctamente indicando que faltaba la clave.



2. Solicitud con API-Key

Posteriormente, se repitió la solicitud, esta vez enviando un API-Key válido. La respuesta fue satisfactoria, mostrando el mensaje “funciona” como prueba de correcto funcionamiento.

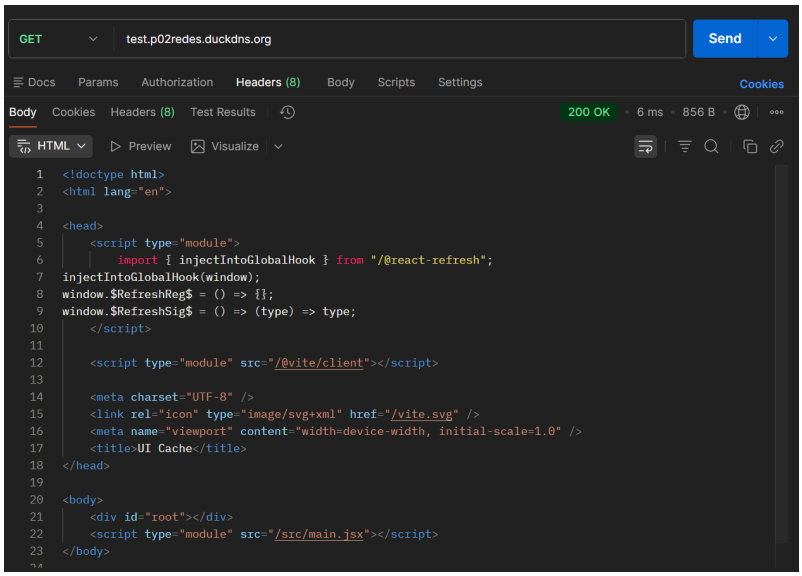
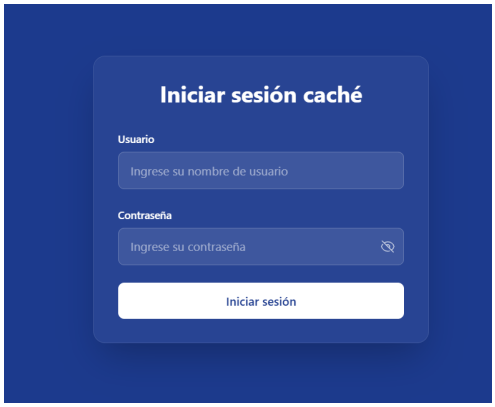


USERPASS

Para validar el flujo de autenticación mediante usuario y contraseña, se realizó la búsqueda del dominio correspondiente desde el navegador, utilizando el subdominio de pruebas (test), el cual, como se mencionó anteriormente, está configurado con autenticación tipo userpass.

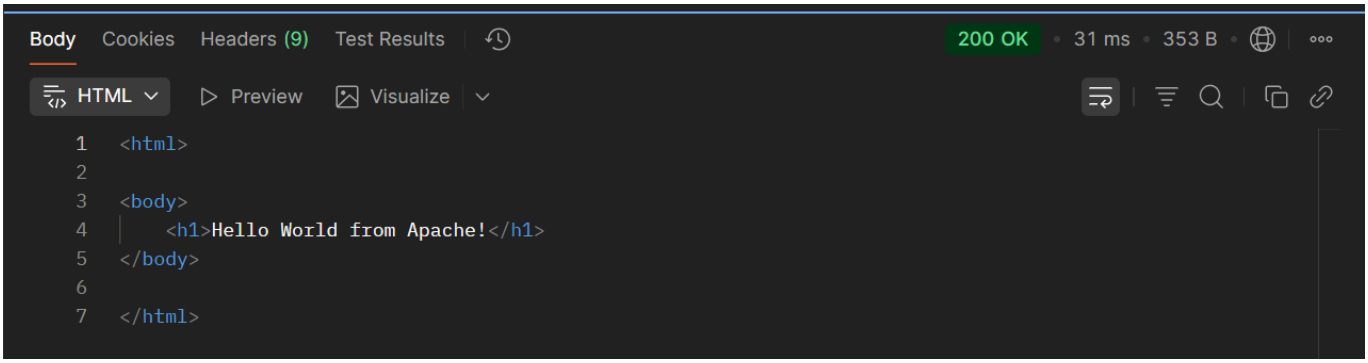
1. Sin session_id

En las siguientes imágenes se observa cómo, al no contar con un session_id, el usuario es redirigido correctamente al login alojado en Vercel, tanto al realizar la prueba desde Postman como visualmente desde el navegador.

Postman	**Navegador**
	

2. Con session id

En este caso, la prueba se realizó utilizando la dirección IP del servidor Apache. En la siguiente imagen se observa que el sistema responde correctamente cuando el usuario envía un session_id válido.



Algoritmos de remplazo

1. MRU: para probar este algoritmo se busco la pagina www.vndb.org(index),v2002 y v10, se volvió a buscar v2002 para que quede como la pagina más reciente. Los archivos guardan [nombre,tamaño,fecha/uso]. Entonces antes del remplazo la cache se ve así:

```
root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  index.html  v10  v2002
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "MRU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 404488,
        "files":        [["index.html", 31383, "2025-11-15 04:22:11"], ["v2002", 271815, "2025-11-15 04:22:17"], ["v10",
101290, "2025-11-15 04:22:16"]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

Luego se buscó v20424, la cual remplazó a la buscaba más reciente porque no habia más espacio en la cache.

```
}root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  index.html  v10  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "MRU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 273511,
        "files":        [["index.html", 31383, "2025-11-15 04:22:11"], ["v10", 101290, "2025-11-15 04:22:16"], ["v20424",
 140838, "2025-11-15 04:23:15"]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

2. LRU: mismo patrón de busqueda que el anterior.

```
root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  index.html  v10  v2002
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "LRU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 404315,
        "files":        [["index.html", 31210, "2025-11-15 04:30:34"], ["v2002", 271815, "2025-11-15 04:30:40"], ["v10",
101290, "2025-11-15 04:30:39"]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

Luego se buscó v20424, la cual remplazó a las dos busquedas mas antiguas, ya que tenia espacio al eliminar la primera mas antigua.

```
}root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  v2002  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "LRU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 412653,
        "files":        [["v2002", 271815, "2025-11-15 04:30:40"], ["v20424", 140838, "2025-11-15 04:31:45"]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

3. FIFO: esta vez el orden de busqueda es v20424, v10, index.

```
root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  index.html  v10  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "FIFO",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 273371,
        "files":        [["v20424", 140838], ["v10", 101290], ["index.html", 31243]]
```

Luego se buscó v2002, lo cual remplazo el primer elemento buscado.

```
}root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  index.html  v10  v2002
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "FIFO",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 404348,
        "files":        [["v10", 101290], ["index.html", 31243], ["v2002", 271815]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

4. LFU: orden de busqueda es v20424, v10, index, pero se realiza 3,2,1 vez respectivamente la busqueda.

```
config.json  index.html  v10  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "LFU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 273678,
        "files":        [["v20424", 140838, 2], ["v10", 101290, 1], ["index.html", 31550, 0]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

Luego se buscó v2002, lo cual remplazo los dos archivos menos consultados para poder ser almacenado.

```
}root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  v2002  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "LFU",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 412653,
        "files":        [["v20424", 140838, 2], ["v2002", 271815, 0]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

5. Random: orden de busqueda es v20424 y v2002.

```
root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  v2002  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "Random",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 412653,
        "files":        [["v20424", 140838], ["v2002", 271815]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

Luego se buscó v10, lo cual remplazo aleatoriamente uno de los dos archivos.

```
}root@app-54c7c8f44-txwdr:/app/www.vndb.org# ls
config.json  v10  v20424
root@app-54c7c8f44-txwdr:/app/www.vndb.org# cat config.json
{
        "total_size":   460800,
        "type": "Random",
        "ip":   "185.189.181.140",
        "file_types":   ["png", "html", "jpg"],
        "used": 242128,
        "files":        [["v20424", 140838], ["v10", 101290]]
}root@app-54c7c8f44-txwdr:/app/www.vndb.org#
```

6. Prueba de tipo: va a consultar por el indice de apache, para esta prueba la configuracion no acepta guardar html en cache. Se hizo esta petición y se recibió lo siguiente:

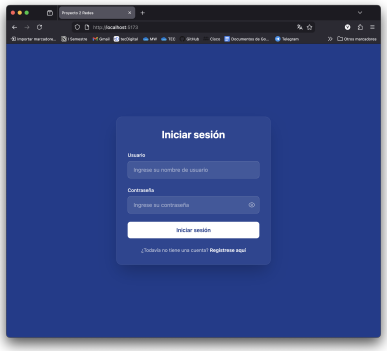
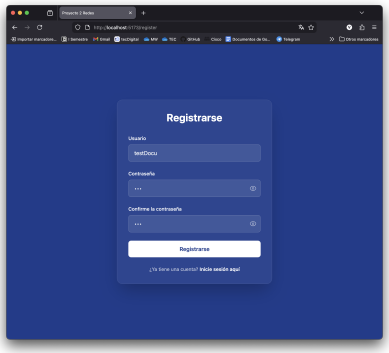
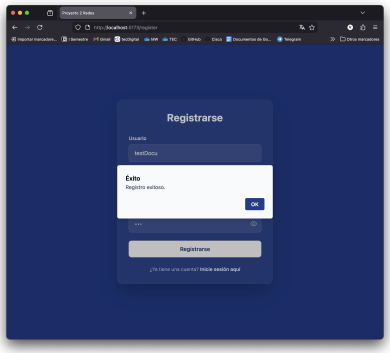
```
 Enviando: GET_FILE apache www /
Recibidos 73 bytes del servidor
b'<html>\n  <body>\n    <h1>Hello World from Apache!</h1>\n  </body>\n</html>\nQ'
```

Como se puede observar se obtuvo lo pedido, a continuación se mostrará que no se guardó en la cache:

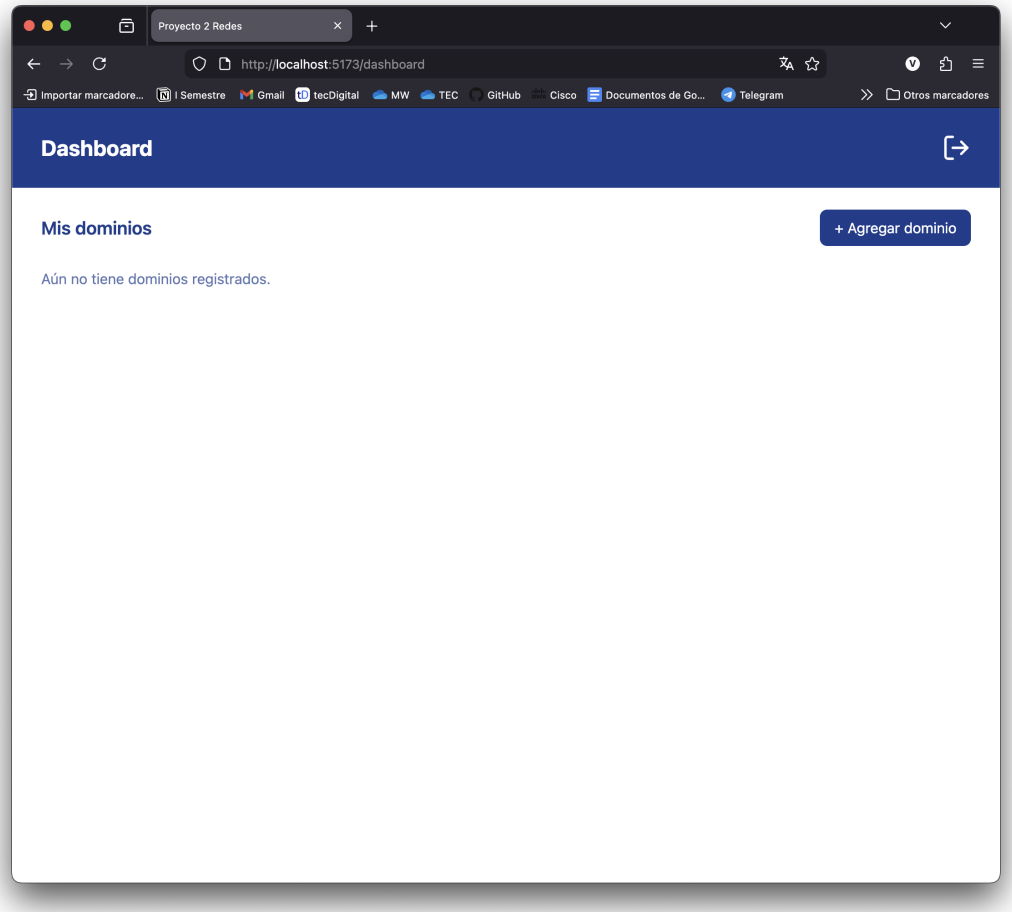
```
root@app-54c7c8f44-9vjl5:/app/www.apache# ls
config.json
root@app-54c7c8f44-9vjl5:/app/www.apache# cat config.json
{
        "total_size":   460800,
        "type": "Random",
        "ip":   "10.1.1.210",
        "file_types":   [],
        "used": 0,
        "files":        []
}root@app-54c7c8f44-9vjl5:/app/www.apache#
```

UI

Al correr la interfaz se muestra el login, si el usuario no está registrado ahí se muestra una opción donde puede registrarse.

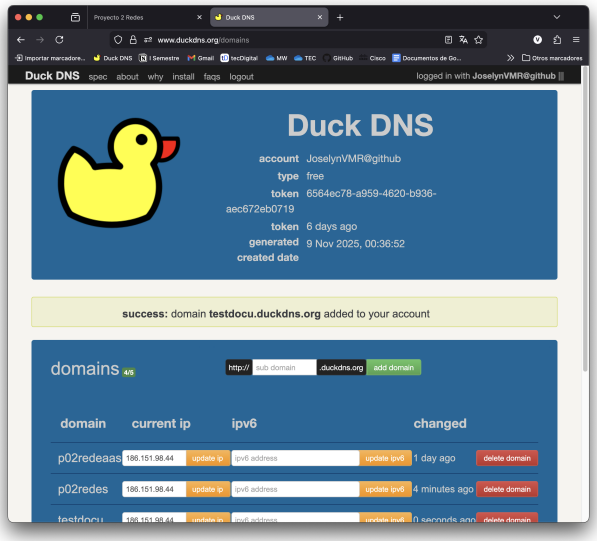
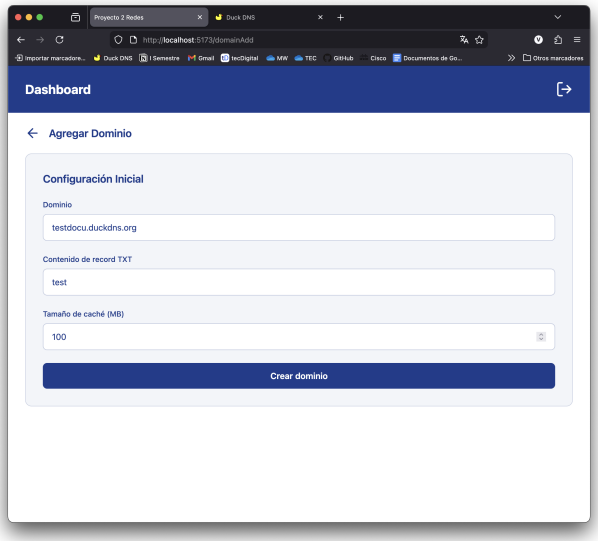
Login	Registro	Registro exitoso
		

Luego de iniciar sesión se lleva al usuario a un dashboard, si es la primera vez que entra este va a estar vacío, tal y como se muestra en la imagen:

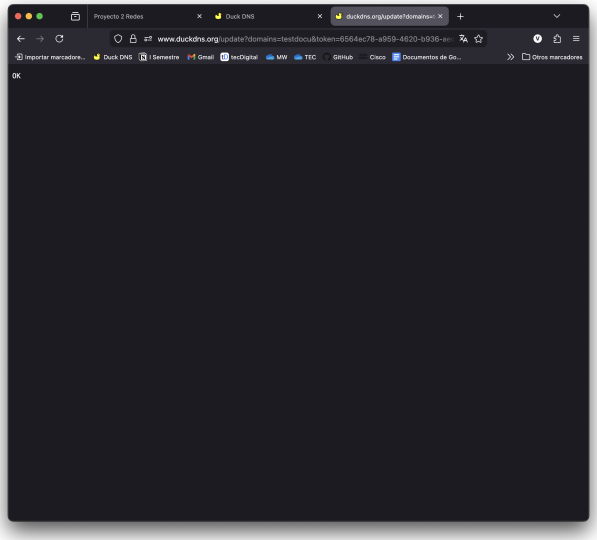


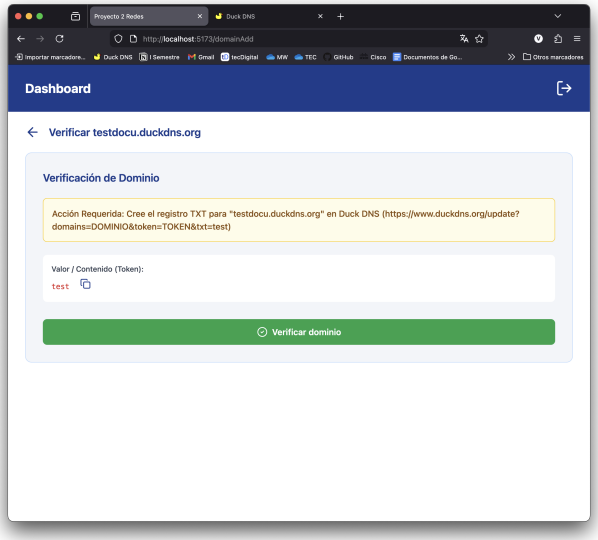
Para la administración de dominios se tiene:

1. Creación de dominio:

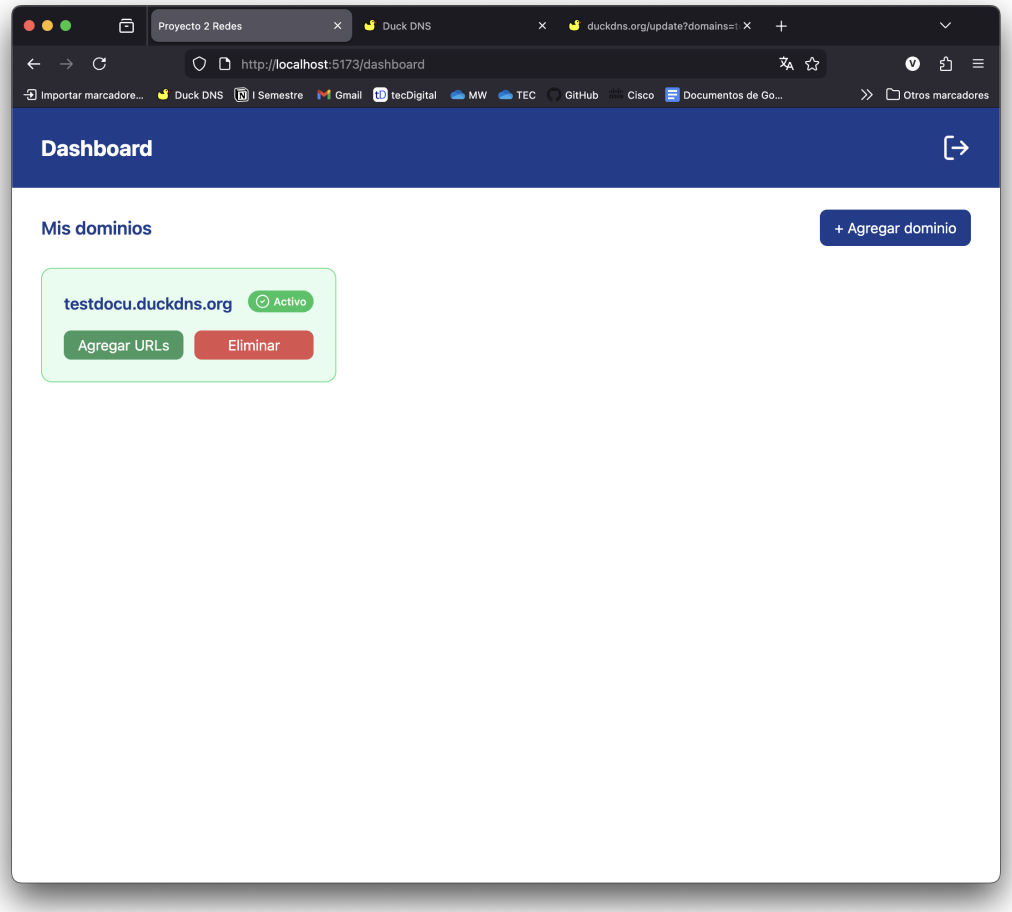
Creación dominio en Duck DNS	Creación de dominio en UI
	
Creación registro TXT en Duck DNS	**Verificación de registro TXT en UI**

Creación registro TXT en Duck DNS **Verificación de registro TXT en UI**

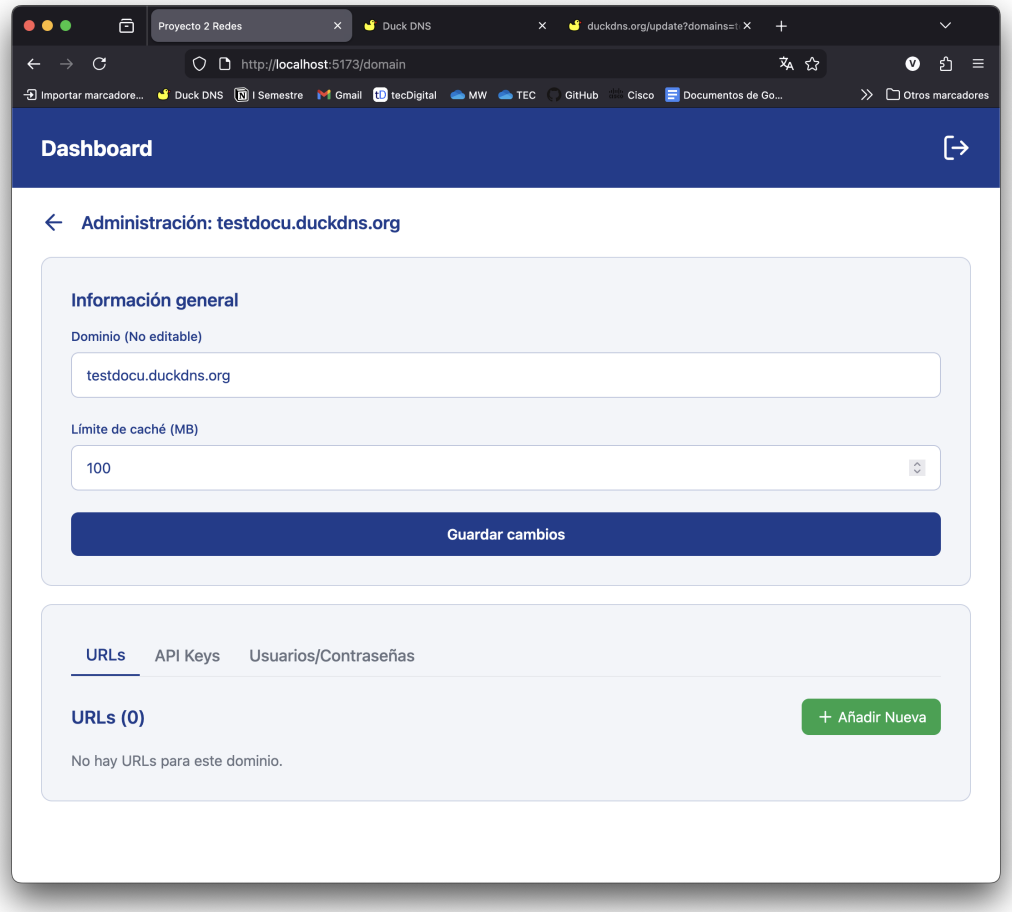




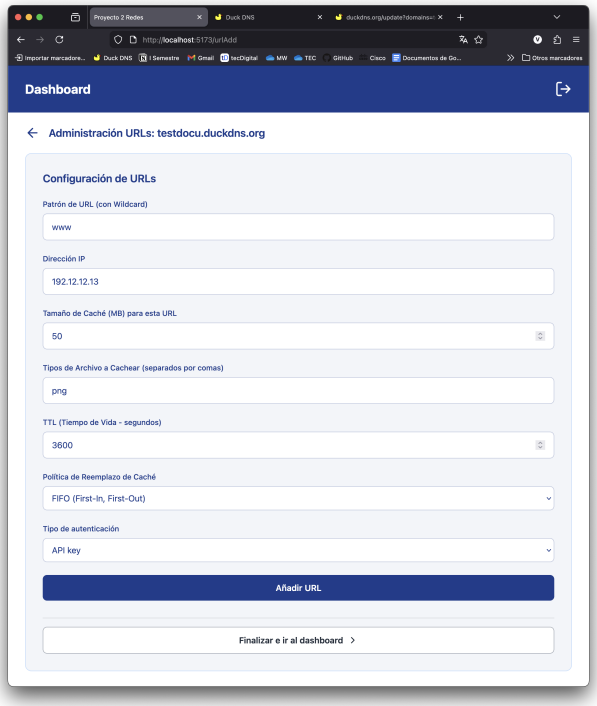
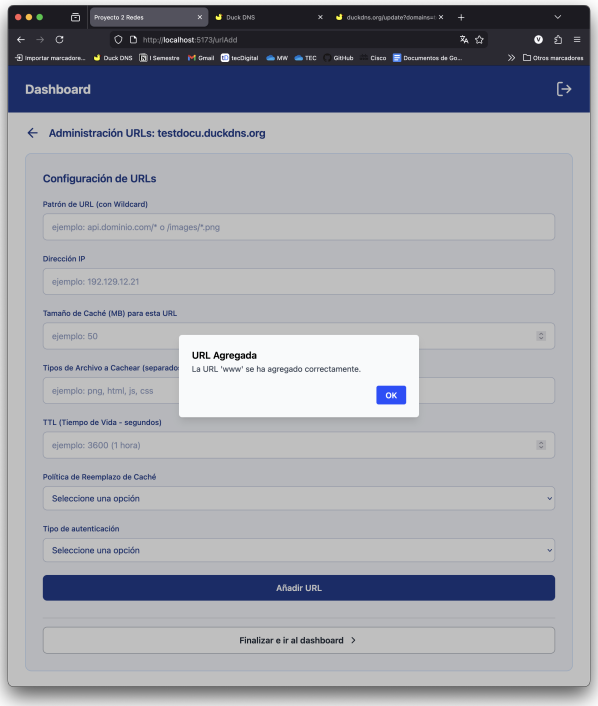
Si la creación es exitosa y se verificó el dominio con el registro TXT el dominio será mostrado en el dashboard con sus botones de acciones:



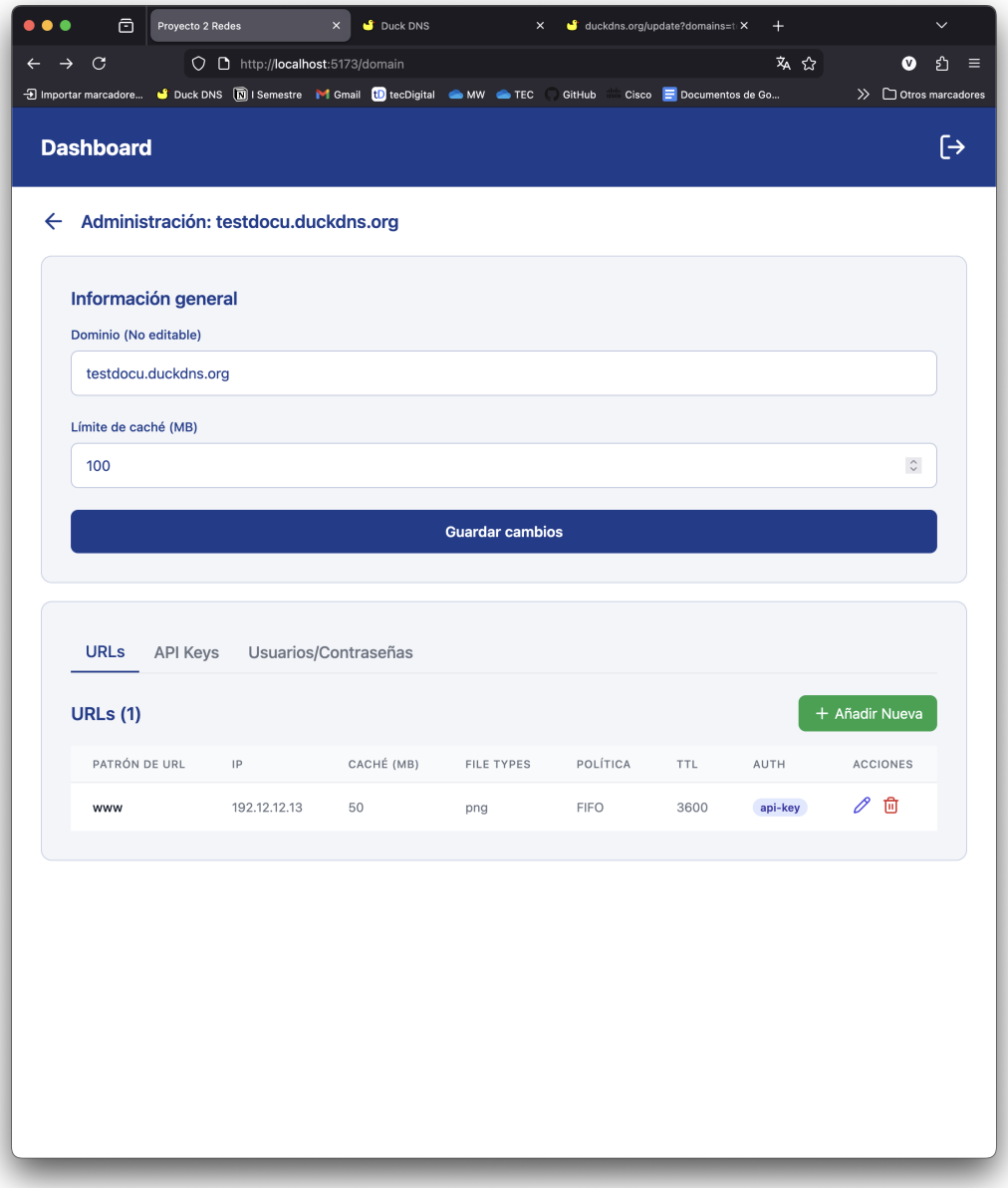
2. Administración de dominio: Al darle doble click al dominio se le lleva al usuario a la página de administración del dominio:



2.1 Administración de urls: Ahora que el dominio está verificado el usuario puede crear urls para ese dominio:

Creación URL	Mensaje exito
	

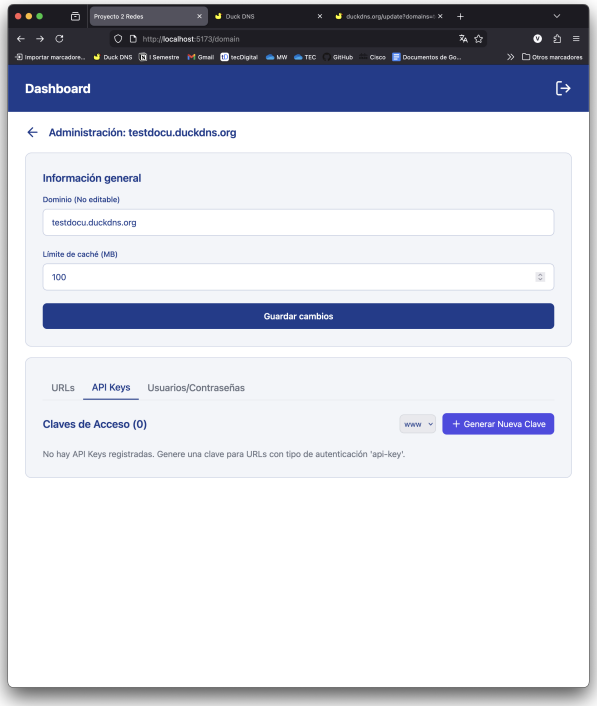
Cuando se crea un registro se actualiza la vista del detalle del dominio:

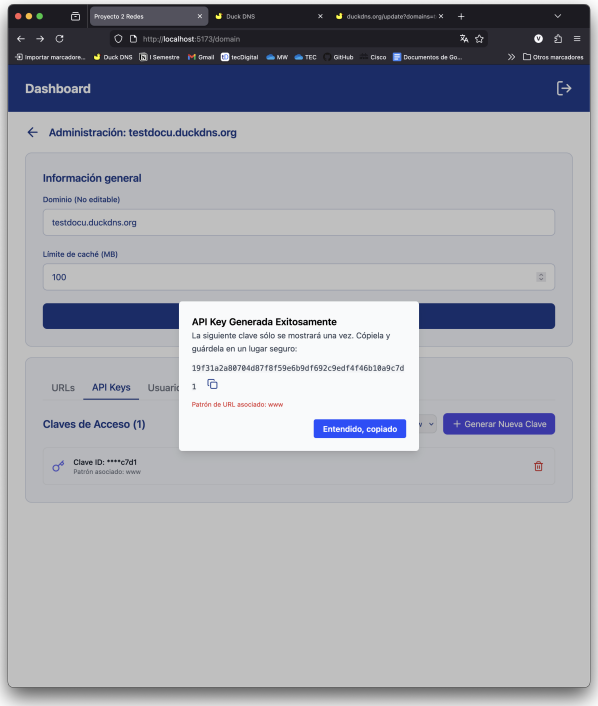


Luego de que se crea un URL, según el tipo de autenticación que se le haya asignado ahora se le puede crear ya sea sus api-keys o userpass. En este caso se especificó que el URL tenía autenticación api-key:

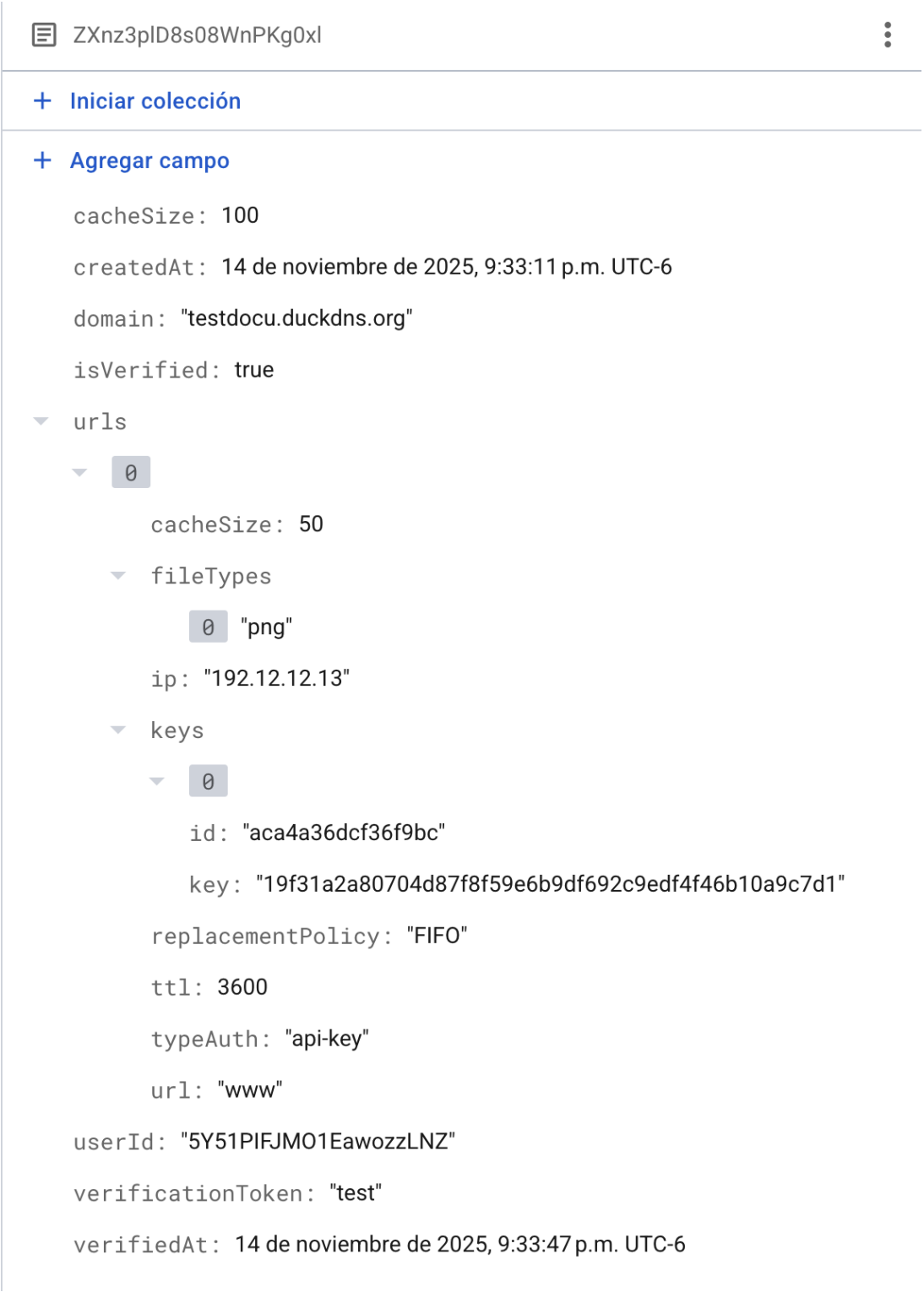
Creación API-key	Mensaje exito

Creación API-key **Mensaje exito**





Toda esta información se guarda en firebase con el siguiente formato:



Conclusiones

1. El DNS Interceptor selecciona correctamente la caché zonal más cercana al usuario, lo cual reduce la latencia y cumple con el objetivo de distribución geográfica del sistema.
2. La integración entre el DNS Interceptor, la API y Firebase funciona adecuadamente, ya que la ubicación del usuario es correctamente comparada con las cachés registradas para determinar el destino óptimo.
3. La caché procesa correctamente las solicitudes HTTP, detectando el dominio, consultando su configuración en la API y aplicando el tipo de autenticación asignado para cada subdominio.

4. La caché procesa correctamente las solicitudes protegidas mediante autenticación por API-Key, validando la presencia y validez de la clave antes de permitir el acceso al recurso solicitado.

5. La caché gestiona adecuadamente las solicitudes que requieren autenticación por usuario y contraseña, identificando la ausencia o invalidez del session_id y redirigiendo al usuario al formulario de login, así como validando sesiones activas para permitir el acceso.

6. La caché respeta las reglas de almacenamiento definidas en Firebase, aplicando TTL, políticas de reemplazo y almacenamiento en disco según lo esperado.

7. La Zonal Cache implementa correctamente los algoritmos de reemplazo configurados (LRU, LFU, FIFO, MRU y Random), aplicándolos de forma coherente para gestionar la permanencia y sustitución de archivos en el almacenamiento local.

8. La API Node.js asegura la comunicación con las caches zonales y la UI mediante la autenticación de API Keys (para las caches) y JWT (para la UI), lo que establece una capa de seguridad esencial en el backend de configuración.

9. La API Node.js cumple su función principal al proveer endpoints para gestionar la totalidad de las configuraciones (dominios, URLs, políticas de cache, TTLs, etc.) directamente en Firebase/Firestore, actuando como el único punto de entrada para modificar la lógica del sistema distribuido.

10. La UI proporciona un flujo completo para el registro, login y manejo de sesiones mediante JWT, asegurando que solo los usuarios autenticados puedan acceder a las funcionalidades de administración.

11. El proceso de verificación de dominio mediante registro TXT utilizando la API de Duck DNS es una solución práctica y funcional que demuestra la propiedad del dominio por parte del usuario, un requisito de seguridad clave para un sistema de cache.

Recomendaciones

1. Se recomienda mantener un formato uniforme para los headers y parámetros HTTP que maneja la caché, con el fin de asegurar coherencia entre los servicios y facilitar integraciones futuras.

2. Es útil registrar los eventos relacionados con la validación del API-Key, especialmente cuando la clave es inválida o no está presente, para mejorar la trazabilidad y el control de seguridad.

3. Se sugiere llevar un registro más detallado del proceso de validación del session_id, principalmente en casos de expiración o problemas con las cookies, para identificar con mayor claridad errores relacionados con las sesiones.

4. Se recomienda monitorear el funcionamiento del TTL y las reglas de almacenamiento, incluyendo pruebas de carga, para verificar que el sistema mantenga un rendimiento estable cuando ocurren muchas expiraciones o reemplazos.

5. Es conveniente analizar el rendimiento de cada algoritmo de reemplazo en distintos escenarios, con el fin de determinar cuál es el más adecuado según el tipo de archivo o el patrón de uso del sistema.

6. Implementar un registro de auditoría en la API que almacene quién (usuario JWT o API Key) y cuándo modificó una configuración clave (políticas de cache, TTLs) en Firebase, para mantener trazabilidad de los cambios.

7. Para mejorar la experiencia del usuario y la seguridad, se recomienda implementar un mecanismo de refresco de tokens para evitar que el usuario deba iniciar sesión repetidamente mientras administra el sistema, actualmente estos tienen un límite de una hora.

8. La verificación de dominio vía Duck DNS debería ser un proceso asíncrono y no bloquear la UI.

9. Se recomienda investigar una herramienta de IA generativa antes de usarla, en este caso el uso de v0 facilitó mucho el trabajo por ir tan alineada a las tecnologías que se estaban usando.

10. Se recomienda mantener la estructura modular de la interfaz, incluyendo la clara división en componentes y páginas, ya que esto facilita la comprensión precisa de la responsabilidad de cada segmento del código y mejora significativamente la mantenibilidad del frontend.